

Project Gamma — System Overview & Architecture

1. What Project Gamma Is

Project Gamma is a **transaction narrative interpretation** system. Its primary responsibility is to **take raw banking narratives and produce a structured, final counterparty resolution** by:

1. Detecting the narrative format
2. Parsing structured fields
3. Extracting payer / payee roles
4. Returning a consistent, machine-usable result

All of this is exposed through one function: **CTPTY()** — the single source of truth. Everything else in the repository exists to support, validate, or operationalize this function.

2. Core Design Principle

One entry point. One responsibility.

```
None
Raw Narrative
  ↓
CTPTY(narrative, amount)
  ↓
{
  parsed: {...},
  ctpty: {
    payer : ,
    payee : ,
    amount:
  }
}
```

```
}  
}
```

- No customer should call parsers directly

If you want a final answer → call `CTPTY()`

3. High-Level Data Flow

We follow this flow.

1. Input

- Narrative (string)
- Tenant_id (int)
- Optional amount
- Optional BAI code (used only as a gate)

2. Format Detection

- Performed internally using heuristics (`detect_format`)
- Determines which parser family applies

3. Parsing

- Parser extracts structured facts (accounts, references, flags, roles)

4. Payer / Payee Extraction

- Centralized in `extract_payer_payee`
- Applies role priority rules and amount direction logic

5. Final Output

- Returned as a unified object via `CTPTY`

4. Repository Structure (Relevant Files Only)

The following table shows the relevant file structure for Project Gamma.

```
Python  
project-gamma/  
|
```

```

├── data/
├── docs/
├── api/
│   └── api.py                # FastAPI wrapper around CTPTY
├── experiments/
│   └── streamlit_app.py      # Streamlit UI (human-in-the-loop)
├── src/
│   └── gamma/
│       ├── counterparty_extraction/ # Extraction logic
│       ├── alias_engine/           # Alias resolution + persistence (see doc)
│       ├── key_engine/             # Canonical key detection (see doc)
│       ├── parsers/                # All narrative parsers (by format)
│       ├── util/                   # Utilities and routing
│       ├── service/                # Public endpoint
│       └── requirements.txt
└── README.md

```

5. CTPTY() — The Canonical Interface

Purpose - `CTPTY()` returns the final interpreted result of a transaction narrative. It combines:

- Format detection
- Parsing
- Payer / payee extraction

None

Conceptual Signature

```
result, fmt = CTPTY(narrative, amount)
```

None

```
{
  "parsed": {
    "...": "format-specific structured fields"
  },
  "ctpty": {
    "payer": "... or null",
    "payee": "... or null",
    "amount": -11700.00
  }
}
```

6. **script.py** — CLI Entrypoint

What it is

- A lightweight developer tool
- Used for:
 - Local testing
 - Debugging parsers
 - Printing CTPTY output

7. API Layer (**api/api.py**)

Purpose : Expose **CTPTY** over HTTP without changing behavior.

Responsibilities

- Validate input
- Apply optional BAI gating
- Call **CTPTY**
- Return structured JSON

Important Rule : The API does not parse on its own. It is a thin wrapper around **CTPTY**.

8. Streamlit UI ([api/stream_app.py](#))

Purpose : Human review and alias confirmation.

What it does

- Accepts narrative input
- Calls **CTPTY**
- Displays parsed + ctpty output
- Resolves payer/payee to canonical aliases
- Persists approved mappings

The UI is a review surface, not a decision engine.

9. Alias Resolution (Referenced)

Alias resolution happens after **CTPTY**.

- Uses normalized text + hybrid fuzzy scoring
- Uses account numbers as hard anchors
- Allows human approval via UI
- Persists aliases for reuse

Refer to the aliasing documentation for more details:

[☰ Alias Resolution Engine: Working and Design](#)

10. Key Engine (Referenced)

The key engine supports parsers by:

- Detecting canonical field labels
- Normalizing key variants across formats
- Improving parser robustness

Refer to the key engine documentation for more details:

[☰ Key Engine — Design & Architecture](#)

11. Parsers (Referenced)

Parsers are:

- Format-specific
- Conservative
- Stateless

They do not assign payer/payee.

Refer to the parsers documentation for more details:

 [Parsers — Design, Responsibilities and Implementation Notes Overview](#)

12. Payer-Payee Extraction

- Once a transaction narrative has been **parsed into structured fields**, the final step is to determine the **payer** and **payee** involved in the transaction. This responsibility is handled by the **Payer–Payee Extraction module**.
- This module does **not** operate on raw narratives. Instead, it consumes the output of the format-specific parsers and applies a strict, rule-based decision flow to identify counterparties safely and consistently.

Refer to this documentation for more details:

 [Payer / Payee Extraction](#)

13. BAI Code Handling (Clarified)

- If provided and unsupported → CTPTY is not executed
- If not provided → CTPTY runs normally

BAI is a gate, not a parser input.

14. Operational Guarantees

- CTPTY is deterministic for a given input
- Alias persistence is explicit and auditable

Maintained by
-Siddharth Gautam